Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

11
텍스트의 유사도

"이 두 글은 얼마나 비슷한가"를 계산하려면 글을 숫자의 나열(벡터)로 바꿔야 합니다. 단어의 횟수를 세어 문서를 벡터로 만들고, 벡터끼리의 유사도를 재어 비슷한 영화를 추천하는 데까지 가 봅니다.

오늘 다루는 것

1. 단어의 표현 방법: 국소 표현과 분산 표현
2. 백 오브 워즈(BoW)
3. DTM과 TF-IDF 행렬
4. 코사인 유사도를 이용한 추천 시스템
5. 유클리드 거리와 자카드 유사도

끝나고 나면 할 수 있는 것

1. 문서 몇 개를 DTM과 TF-IDF 행렬로 직접 계산해 만들 수 있다.
2. 코사인 유사도의 식을 읽고, 문서 길이에 영향을 받지 않는 이유를 설명할 수 있다.
3. 횟수를 세는 표현의 한계 두 가지(희소 표현, 뜻을 모름)를 예를 들어 설명할 수 있다.

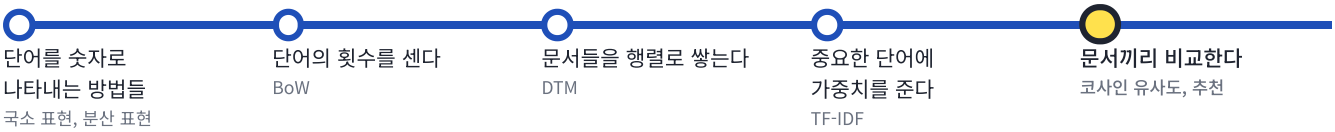
2026년 10월 12일 월요일 14:30 (10/12 자료 1/2)
발표자 설명, 코랩 실습, 퀴즈 돌아가며 풀기 순서로 진행합니다. 실습 2만 `!pip install konlpy` 가 필요하고, 나머지는 코랩에 기본으로 있는 사이킷런과 넘파이를 씁니다.

11

텍스트의 유사도

09장에서 문장을 정수의 줄로 바꿨습니다. 하지만 "영화 = 2, 정말 = 3" 같은 번호로는 두 글이 비슷한지 알 수 없습니다. 번호 2와 3이 가깝다고 뜻이 가까운 것이 아니기 때문입니다. 글끼리 비교하려면 글 하나를 같은 길이의 벡터 하나로 바꿔야 합니다. 가장 단순한 방법은 "어떤 단어가 몇 번 나왔는지" 세는 것입니다. 세어서 만든 벡터로 문서를 비교하고, 그 방법이 어디까지 통하는지 확인합니다.

왜 배우나: 오늘의 학습 경로



개념마다 정의와 식을 먼저 세우고, 과일에 대한 짧은 문서 네 개에 대입해 손으로 계산합니다.
마지막에는 영화 줄거리로 비슷한 영화를 추천합니다. 오늘 확인할 "횟수 세기의 한계"가 다음 자료인 12장 워드 임베딩이 필요한 이유입니다.

11-01 단어의 표현 방법

핵심 한 줄

단어를 숫자로 나타내는 방법은, 단어 자체에 값을 붙이는 국소 표현과 주변 단어를 참고해 뜻을 벡터에 담는 분산 표현으로 나뉜다.

왜 배우나

11장과 12장에서 단어와 문서를 숫자로 바꾸는 방법이 여러 개 나옵니다. 하나씩 보기 전에 전체 지도를 먼저 봅니다. 지금 배우는 방법이 지도의 어디에 있고 무엇이 부족한지 알면, 다음 방법이 왜 나왔는지 이해하기 쉽습니다.

1) 국소 표현과 분산 표현

정의

- 국소 표현(local representation): 해당 단어 **그 자체만** 보고 특정 값을 대응시켜 표현하는 방법. 이산 표현(discrete representation)이라고도 합니다. 단어마다 번호를 붙이는 정수 인코딩, 원-핫 벡터, 횟수를 세는 BoW가 여기에 속합니다.
- 분산 표현(distributed representation): 그 단어의 **주변 단어를 참고해** 표현하는 방법. 단어의 뜻이 벡터의 여러 칸에 나뉘어(분산되어) 담깁니다. 연속 표현(continuous representation)이라고도 하며, 12장의 워드 임베딩이 여기에 속합니다.

둘의 가장 큰 차이는 "비슷한 단어가 비슷한 값을 갖는가"입니다. 국소 표현은 단어마다 따로 값을 정하므로 값에 단어의 뜻이 들어 있지 않습니다. 분산 표현은 비슷한 자리에 쓰이는 단어들이 비슷한 벡터를 갖도록 만들어, 벡터만 보고도 뜻이 가까운지 알 수 있습니다.

예시 적용

"강아지", "고양이", "자동차"를 두 가지 방법으로 나타내고 서로 얼마나 비슷한지(코사인 유사도, 11-04) 재 봤습니다(실습 1).

표현	강아지	고양이	자동차	강아지-고양이	강아지-자동차
국소 표현 (원-핫)	[1, 0, 0]	[0, 1, 0]	[0, 0, 1]	0.0	0.0
분산 표현 (설명용 값)	[0.9, 0.8, 0.1]	[0.8, 0.9, 0.2]	[0.1, 0.0, 0.9]	0.99	0.16

원-핫 벡터에서는 어떤 두 단어를 골라도 유사도가 0입니다. 강아지가 고양이와 가까운지 자동차와 가까운지 알 방법이 없습니다. 분산 표현에서는 칸마다 뜻이 조금씩 나뉘어 담겨 있어서(여기서는 설명을 위해 [귀여움, 동물, 탈것]이라고 직접 정했습니다) 강아지와 고양이가 가깝게 나옵니다. 실제 분산 표현은 사람이 값을 정하지 않고 글에서 학습합니다(12장).

2) 단어 표현의 분류

정의

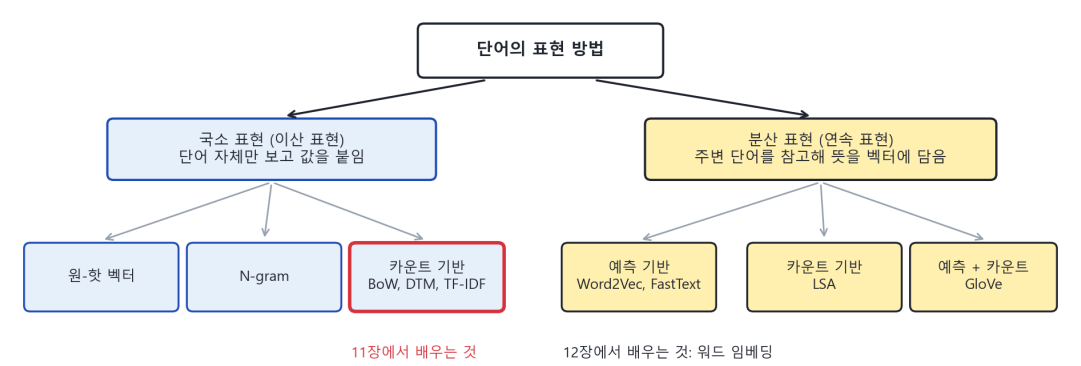


그림 1 교재의 분류. 왼쪽 파란 상자가 국소 표현, 오른쪽 노란 상자가 분산(연속) 표현이다. 빨간 테두리가 이번 장의 내용

- **국소 표현**: 원-핫 벡터(05장, 12-01), N-gram(10장), 그리고 횟수를 세는 **Bag of Words**와 그 확장인 DTM, TF-IDF(이번 장)

- **분산(연속) 표현**: 다음 단어를 예측하게 하면서 학습하는 Word2Vec과 FastText, 횟수를 바탕으로 하는 LSA, 두 방식을 함께 쓰는 GloVe(12장)

예시 적용

지금까지 쓴 방법을 이 지도에 놓아 보면, 07-03의 원-핫 벡터와 09-05의 정수 인코딩은 모두 국소 표현입니다. 이번 장의 BoW도 국소 표현이지만, 단어 하나가 아니라 **문서 하나**를 벡터로 만든다는 점이 다릅니다. 그래서 문서끼리 비교할 수 있게 됩니다.

비유

사물함 번호와 자기소개서

학생에게 사물함 번호 17번을 줬다고 그 학생이 어떤 사람인지 알 수는 없습니다. 17번과 18번이 친한 사이라는 뜻도 아닙니다. 반면 자기소개서에는 취미, 성격, 좋아하는 과목이 항목별로 적혀 있어서, 두 사람의 소개서를 나란히 놓으면 얼마나 비슷한지 알 수 있습니다.

사물함 번호	= = 국소 표현 (번호, 원-핫)
자기소개서의 여러 항목	= = 분산 표현 (벡터의 여러 칸)
소개서를 나란히 비교	= = 벡터의 유사도

실습 1

국소 표현과 분산 표현에서 단어의 유사도

```python
import numpy as np

def cos_sim(a, b):
    """두 벡터가 얼마나 같은 방향인지 (11-04에서 자세히)"""
    return a @ b / (np.linalg.norm(a) * np.linalg.norm(b))

words = ["강아지", "고양이", "자동차"]

# 1) 국소 표현: 단어마다 번호(정수 인코딩)나 원-핫 벡터를 붙인다
word_to_index = {w: i for i, w in enumerate(words)}
onehot = np.eye(len(words))
print("정수 인코딩:", word_to_index)
for w in words:
    print(f"  원-핫 {w}: {onehot[word_to_index[w]]}")
print("원-핫 유사도  강아지-고양이:", cos_sim(onehot[0], onehot[1]),
      " 강아지-자동차:", cos_sim(onehot[0], onehot[2]))

# 2) 분산 표현: 뜻이 여러 칸에 나뉘어 담긴 벡터 (설명용으로 직접 정한 값)
#    칸의 뜻을 [귀여움, 동물, 탈것]이라고 하자
dense = {"강아지": np.array([0.9, 0.8, 0.1]),
         "고양이": np.array([0.8, 0.9, 0.2]),
         "자동차": np.array([0.1, 0.0, 0.9])}
print("분산 표현 유사도  강아지-고양이:",
      round(cos_sim(dense["강아지"], dense["고양이"]), 2),
      " 강아지-자동차:", round(cos_sim(dense["강아지"], dense["자동차"]), 2))
```

실행 결과

```
정수 인코딩: {'강아지': 0, '고양이': 1, '자동차': 2}
  원-핫 강아지: [1. 0. 0.]
  원-핫 고양이: [0. 1. 0.]
  원-핫 자동차: [0. 0. 1.]
원-핫 유사도  강아지-고양이: 0.0  강아지-자동차: 0.0
분산 표현 유사도  강아지-고양이: 0.99  강아지-자동차: 0.16
```

직접 해 보기 `dense`에 "버스": [0.0, 0.0, 0.8]을 추가하고 자동차와의 유사도를 구해 보세요. 원-핫이었다면 얼마였을까요?

확인 문제

11-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 원-핫 벡터는 국소 표현이다. (O / X) 이름 _____

2 국소 표현에서는 뜻이 비슷한 단어가 비슷한 값을 갖는다. (O / X) 이름 _____

3 주변 단어를 참고해 단어의 뜻을 벡터에 담는 방법은? 이름 _____

① 국소 표현 ② 분산 표현 ③ 정수 인코딩

4 이번 장에서 배우는 Bag of Words는 어디에 속하나요? 이름 _____

① 국소 표현 (카운트 기반) ② 분산 표현 (예측 기반) ③ 어느 쪽도 아님

5 국소 표현을 다른 말로 _____ 표현이라고도 한다. 이름 _____

6 실습에서 원-핫 벡터로 잰 강아지와 고양이의 유사도는 _____이었다. 이름 _____

7 국소 표현과 분산 표현을 각각 정의하고, 강아지·고양이·자동차 예시로 차이를 설명해 보세요. 이름 _____

원문: 11-01 단어의 표현 방법 — https://wikidocs.net/217499 (교재의 puppy, cute, lovely 예를 강아지, 고양이, 자동차로 바꾸고 유사도 실습을 추가했습니다)

11-02 백 오브 워즈(Bag of Words)

핵심 한 줄 **Bag of Words는 단어의 순서를 버리고 각 단어가 나온 횟수만 세어, 문서 하나를 단어 집합 크기의 벡터 하나로 나타내는 방법이다.**

왜 배우나 문서는 길이가 제각각이라 그대로는 비교할 수 없습니다. BoW는 어떤 길이의 문서든 **같은 길이의 벡터**로 바꿔 주는 가장 단순한 방법입니다. "어떤 단어가 많이 나왔는가"만으로도 문서의 주제는 꽤 잘 드러나므로, 문서 분류와 유사도 계산의 출발점이 됩니다.

1) Bag of Words의 정의

정의

Bag of Words(BoW)는 단어들의 **순서는 전혀 고려하지 않고 출현 빈도에만 집중**하는 텍스트 표현 방법입니다. 문서의 단어를 모두 가방에 넣고 흔들어 섞은 뒤, 가방 안에 어떤 단어가 몇 개 있는지만 세는 것과 같습니다. 만드는 과정은 두 단계입니다.

1. 단어마다 고유한 번호를 붙입니다(단어 집합 만들기, 09-05).
2. 번호 자리에 그 단어가 문서에서 나온 횟수를 적은 벡터를 만듭니다.

- 벡터의 길이는 단어 집합의 크기 V입니다. 문서가 길든 짧든 길이는 V로 같습니다.
- 여러 문서를 비교하려면 **모든 문서를 합쳐 만든 하나의 단어 집합**으로 각 문서의 BoW를 만들어야 합니다. 그래야 같은 칸이 같은 단어를 뜻합니다.
- **쓰임**: 어떤 단어가 얼마나 나왔는지로 문서의 성격을 판단하는 일, 곧 문서 분류와 문서 사이의 유사도 계산
- **한계**: 순서를 버리므로, 단어는 같고 순서만 다른 문장을 구분하지 못합니다.

예시 적용 문서 1 "정부가 발표하는 물가상승률과 소비자가 느끼는 물가상승률은 다르다."를 형태소로 자르면 토큰 12개가 나오고, 서로 다른 토큰은 10개입니다(실습 2).

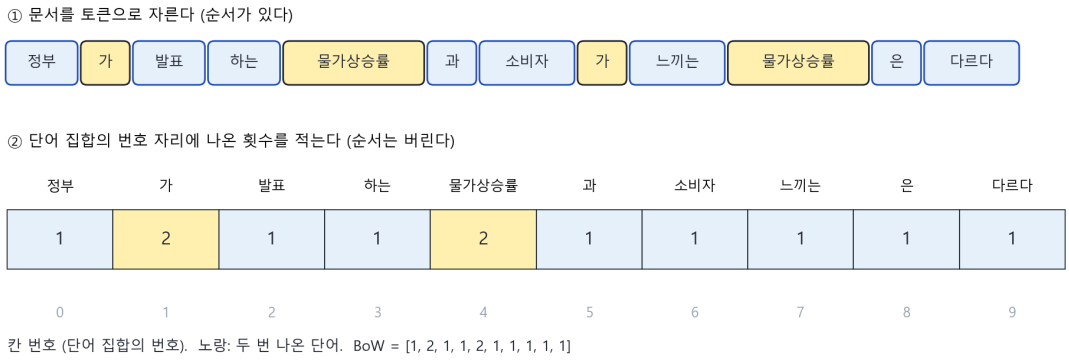


그림 2 토큰의 순서를 버리고, 단어 집합의 번호 자리에 횟수를 적는다. "가"와 "물가상승률"이 두 번씩 나와 그 칸이 2다

문서 2 "소비자는 주로 소비하는 상품을 기준으로 물가상승률을 느낀다."와 비교하려면 두 문서를 합친 단어 집합(18개)으로 다시 만듭니다.

문서	BoW (길이 18)
문서 1	`[1, 2, 1, 1, 2, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0]`
문서 2	`[0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 2, 1, 1, 1]`

"물가상승률"은 4번 칸입니다. 문서 1에서 2번, 문서 2에서 1번 나와 값이 각각 2와 1입니다. 두 벡터가 3번("하는"), 4번("물가상승률"), 6번("소비자") 칸에서 함께 0이 아니라는 것이 두 문서가 비슷한 이야기를 한다는 단서가 됩니다.

순서를 버린 대가. "dog bites man"(개가 사람을 문다)과 "man bites dog"(사람이 개를 문다)의 BoW는 둘 다 [1, 1, 1]로 똑같습니다(실습 3). 뜻은 정반대인데 BoW로는 구분할 수 없습니다.

2) 사이킷런의 CountVectorizer

정의

- CountVectorizer: 사이킷런이 제공하는 클래스로, 단어 집합 만들기와 BoW 만들기를 한 번에 해 줍니다. `fit_transform(문서 리스트)` 는 BoW를, `vocabulary_` 는 단어와 번호의 짝을 돌려줍니다.
- **기본 동작**: 모두 소문자로 바꾸고, 띄어쓰기와 구두점으로 자르며, **두 글자 이상**인 것만 토큰으로 봅니다. 번호는 알파벳 순서로 붙습니다.
- **불용어 제거**: `stop_words` 에 직접 만든 리스트를 주거나, `"english"` 를 주면 사이킷런의 영어 불용어 목록을 씁니다. NLTK의 목록을 넘겨도 됩니다.

왜 한국어에는 그대로 쓰면 안 될까? 띄어쓰기로만 자르기 때문입니다. 09-01에서 본 것처럼 한국어는 조사가 붙어 같은 단어가 서로 다른 토큰이 됩니다. 한국어는 형태소 분석기로 먼저 토큰화한 뒤에 세어야 합니다.

왜 BoW에서 불용어를 뺄까? BoW는 "많이 나온 단어가 그 문서를 대표한다"고 보는 방법입니다. the, is 같은 불용어는 어느 문서에서나 많이 나오므로 문서의 특징을 흐립니다. 벡터의 길이 V도 줄어듭니다.

예시 적용

"you know I want your love. because I love you."를 넣으면 BoW는 [1, 1, 2, 1, 2, 1], 단어 집합은 because = 0, know = 1, love = 2, want = 3, you = 4, your = 5입니다(실습 3). love와 you가 2번씩 나왔고, 한 글자인 I는 토큰에서 빠졌습니다.

문서 1을 그대로 넣으면 "물가상승률과"와 "물가상승률은"이 서로 다른 단어로 세어집니다. 실습 2에서 Okt로 자른 결과와 비교해 보세요.

"Family is not an important thing. It's everything."에서 불용어를 빼면, 직접 정한 다섯 단어를 뺀 경우 [everything, family, important, it, thing]이 남고, 사이킷런의 목록을 쓰면 [family, important, thing]만 남습니다. 목록에 따라 결과가 달라지고, 09-03에서 본 것처럼 not도 함께 빠집니다.

비유

장바구니 영수증

영수증에는 무엇을 몇 개 샀는지만 적혀 있고, 어떤 순서로 카트에 담았는지는 없습니다. 그래도 영수증만 보면 "이 집은 아기가 있구나", "캠핑 가는구나"를 짐작할 수 있습니다. 두 집의 영수증을 나란히 놓으면 장 보는 습관이 얼마나 비슷한지도 알 수 있습니다.

영수증 한 장	= = 문서 하나의 BoW
품목마다의 수량	= = 단어마다의 횟수
담은 순서는 없음	= = 단어의 순서를 버림
마트의 전체 품목표	= = 단어 집합

실습 2

BoW 직접 만들기 (한국어)

```
# 코랩에서는 먼저:  !pip install konlpy
from konlpy.tag import Okt

okt = Okt()

def tokenize(document):
    return okt.morphs(document.replace(".", ""))      # 마침표를 지우고 형태소로

def build_vocab(documents):
    """(1) 단어마다 고유한 번호를 붙인다 = 단어 집합"""
    vocab = {}
    for doc in documents:
        for word in tokenize(doc):
            if word not in vocab:
                vocab[word] = len(vocab)
    return vocab

def bag_of_words(document, vocab):
    """(2) 번호 자리에 그 단어가 나온 횟수를 적는다 = BoW"""
    bow = [0] * len(vocab)
    for word in tokenize(document):
        bow[vocab[word]] += 1
    return bow

doc1 = "정부가 발표하는 물가상승률과 소비자가 느끼는 물가상승률은 다르다."
```

```
doc2 = "소비자는 주로 소비하는 상품을 기준으로 물가상승률을 느낀다."

vocab1 = build_vocab([doc1])
print("문서1의 단어 집합:", vocab1)
print("문서1의 BoW      :", bag_of_words(doc1, vocab1))

# 두 문서를 비교하려면 같은 단어 집합으로 BoW를 만들어야 한다
vocab = build_vocab([doc1, doc2])
print("\n합친 단어 집합의 크기:", len(vocab))
print("문서1의 BoW:", bag_of_words(doc1, vocab))
print("문서2의 BoW:", bag_of_words(doc2, vocab))
i = vocab["물가상승률"]
n1, n2 = bag_of_words(doc1, vocab)[i], bag_of_words(doc2, vocab)[i]
print(f"'물가상승률'은 {i}번 칸: 문서1에서 {n1}번, 문서2에서 {n2}번")
```

```
실행 결과
문서1의 단어 집합: {'정부': 0, '가': 1, '발표': 2, '하는': 3, '물가상승률': 4, '과': 5, '소비자': 6, '느끼는': 7,
'은': 8, '다르다': 9}
문서1의 BoW      : [1, 2, 1, 1, 2, 1, 1, 1, 1, 1]

합친 단어 집합의 크기: 18
문서1의 BoW: [1, 2, 1, 1, 2, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0]
문서2의 BoW: [0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 2, 1, 1, 1]
'물가상승률'은 4번 칸: 문서1에서 2번, 문서2에서 1번
```

직접 해 보기 `tokenize` 에서 조사 "가", "은", "과", "는", "을"을 빼도록 고치면 단어 집합의 크기가 몇으로 줄어드나요?

실습 3 CountVectorizer로 BoW 만들기

```
from sklearn.feature_extraction.text import CountVectorizer

# 1) 사이킷런의 CountVectorizer: 단어 집합과 BoW를 한 번에
corpus = ["you know I want your love. because I love you."]
vector = CountVectorizer()
print("BoW      :", vector.fit_transform(corpus).toarray())
print("단어 집합:", vector.vocabulary_)        # I는 한 글자라서 빠진다

# 2) 순서를 무시한다: 뜻이 반대인 두 문장의 BoW가 같다
pair = ["dog bites man", "man bites dog"]
vector = CountVectorizer()
print("\n", vector.fit(pair).get_feature_names_out())
print(" dog bites man →", vector.transform(pair).toarray()[0])
print(" man bites dog →", vector.transform(pair).toarray()[1])

# 3) 한국어에 그대로 쓰면: 띄어쓰기로만 잘라서 조사가 붙은 채로 센다
ko = ["정부가 발표하는 물가상승률과 소비자가 느끼는 물가상승률은 다르다."]
vector = CountVectorizer()
vector.fit(ko)
print("\n한국어:", sorted(vector.vocabulary_))

# 4) 불용어를 빼고 만들기
text = ["Family is not an important thing. It's everything."]
mine = CountVectorizer(stop_words=["the", "a", "an", "is", "not"])
mine.fit(text)
print("\n직접 정한 불용어:", sorted(mine.vocabulary_))
builtin = CountVectorizer(stop_words="english")
builtin.fit(text)
print("사이킷런 불용어 :", sorted(builtin.vocabulary_))
```

```
실행 결과
BoW      : [[1 1 2 1 2 1]]
단어 집합: {'you': 4, 'know': 1, 'want': 3, 'your': 5, 'love': 2, 'because': 0}

 ['bites' 'dog' 'man']
 dog bites man → [1 1 1]
 man bites dog → [1 1 1]

한국어: ['느끼는', '다르다', '물가상승률과', '물가상승률은', '발표하는', '소비자가', '정부가']

직접 정한 불용어: ['everything', 'family', 'important', 'it', 'thing']
사이킷런 불용어 : ['family', 'important', 'thing']
```

직접 해 보기 `CountVectorizer(ngram_range=(1, 2))` 로 바꾸면 "dog bites man"과 "man bites dog"의 BoW가 달라집니다. 10-02의 bigram을 떠올려 이유를 설명해 보세요.

11-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 Bag of Words는 단어의 순서를 고려한다. (O / X) 이름 _____

2 CountVectorizer는 한국어도 형태소 단위로 알아서 잘라 준다. (O / X) 이름 _____

3 BoW 벡터의 길이는 무엇과 같은가요? 이름 _____

① 문서의 글자 수 ② 단어 집합의 크기 ③ 문장의 수

4 "dog bites man"과 "man bites dog"의 BoW가 같은 이유는? 이름 _____

① 순서를 버리고 횟수만 세기 때문에 ② 불용어를 지웠기 때문에 ③ 소문자로 바꿨기 때문에

5 실습에서 문서 1의 BoW에서 "물가상승률" 칸의 값은 _____였다. 이름 _____

6 CountVectorizer는 기본으로 길이가 _____글자 이상인 것만 토큰으로 본다. 이름 _____

7 Bag of Words를 정의하고, 장바구니 영수증에 빗대어 장점과 한계를 설명해 보세요. 이름 _____

원문: 11-02 백 오브 워즈(Bag of Words) 방법 — https://wikidocs.net/217500

11-03 DTM과 TF-IDF 행렬

핵심 한 줄

DTM은 문서마다의 BoW를 쌓은 행렬이고, TF-IDF는 여러 문서에 흔히 나오는 단어의 값은 낮추고 특정 문서에만 나오는 단어의 값은 높인 행렬이다.

왜 배우나

문서를 비교하려면 모든 문서를 같은 틀에 놓아야 합니다. 그 틀이 DTM입니다. 그런데 횟수만 세면 어디에나 나오는 단어가 큰 값을 차지해 문서의 차이가 묻힙니다. TF-IDF는 "이 단어가 이 문서에서 얼마나 중요한가"를 계산해 그 문제를 덜어 줍니다. 검색 엔진과 추천에서 지금도 쓰이는 방법입니다.

1) 문서 단어 행렬(DTM)

정의

- 문서 단어 행렬(Document-Term Matrix, DTM): 여러 문서에서 각 단어가 나온 횟수를 행렬로 나타낸 것. **행이 문서, 열이 단어**이고, 문서마다의 BoW를 한 줄씩 쌓은 것입니다. 문서가 n개, 단어 집합의 크기가 V이면 모양은 $(n,\ V)$입니다. 각 행을 **문서 벡터**라고 합니다.
- **한계 1. 희소 표현**: 문서 하나에 나오는 단어는 전체 단어 중 극히 일부라서 대부분의 값이 0입니다. 이런 벡터와 행렬을 희소 벡터, 희소 행렬이라고 합니다. V가 수만이 되면 저장 공간과 계산이 많이 듭니다. 그래서 09장의 전처리로 단어 집합을 줄이는 일이 중요합니다.
- **한계 2. 단순 빈도**: the처럼 어느 문서에나 자주 나오는 단어가 큰 값을 가집니다. 두 문서에 the가 많다고 비슷한 문서인 것은 아닙니다.

예시 적용

과일에 대한 짧은 문서 네 개를 띄어쓰기로 잘라 DTM을 만듭니다(실습 4). 서로 다른 단어가 9개이므로 모양은 (4, 9)입니다.

DTM: 행 = 문서, 열 = 단어, 값 = 나온 횟수. 모양 (4, 9)

	과일이	길고	노란	먹고	바나나	사과	싶은	저는	좋아요
문서1: 먹고 싶은 사과	0	0	0	1	0	1	1	0	0
문서2: 먹고 싶은 바나나	0	0	0	1	1	0	1	0	0
문서3: 길고 노란 바나나 바나나	0	1	1	0	2	0	0	0	0
문서4: 저는 과일이 좋아요	1	0	0	0	0	0	0	1	1

36칸 중 24칸이 0이다 (희소 표현). 노랑: 2번 나온 '바나나'

그림 3 문서 네 개의 DTM. 한 행이 문서 하나의 BoW다. 문서 3의 "바나나"만 2이고, 36칸 중 24칸이 0이다

문서 4개에 단어 9개뿐인데도 3분의 2가 0입니다. 교재의 영화 줄거리 2만 편으로 만든 행렬은 (20000, 47487)로, 한 줄에 4만 7천 칸이 있고 거의 모두 0입니다.

2) TF-IDF

정의

TF-IDF(Term Frequency-Inverse Document Frequency, 단어 빈도-역 문서 빈도)는 DTM의 각 값에 그 단어의 중요도를 가중치로 곱한 것입니다. 문서를 d, 단어를 t, 문서의 총 개수를 n이라고 하면 세 가지 값으로 만듭니다.

- $tf(d,t)$: 문서 d에서 단어 t가 나온 횟수. **DTM의 값 그대로**입니다.
- $df(t)$: 단어 t가 나온 **문서의 수**. 한 문서 안에서 몇 번 나왔는지는 세지 않습니다. 한 문서에 100번 나와도 그 문서 하나로 1입니다.
- $idf(t)$: $df(t)$에 반비례하는 값. 여러 문서에 나오는 흔한 단어일수록 작아집니다.

$$idf(t) = \log\left(\frac{n}{1+df(t)}\right) \qquad \text{tf-idf}(d,t) = tf(d,t) \times idf(t)$$

말로 하면: "이 문서에서 많이 나왔는가(TF)" × "다른 문서에는 잘 안 나오는가(IDF)". 둘 다 클 때, 곧 **이 문서에서만 자주 나오는 단어**일 때 값이 큽니다. 모든 문서에 나오는 불용어는 TF가 커도 IDF가 작아 값이 낮아집니다.

왜 분모에 1을 더할까? 어떤 문서에도 나오지 않은 단어는 $df = 0$이라 0으로 나누게 됩니다. 이를 막기 위한 것입니다.

왜 로그를 씌울까? 로그 없이 n/df를 쓰면 희귀한 단어의 값이 지나치게 커집니다. 문서가 100만 개일 때 1개 문서에만 나온 단어는 100만, 1,000개 문서에 나온 단어는 1,000이 되어 1,000배나 차이 납니다. 로그를 씌우면 6과 3으로 2배 차이가 됩니다. 로그는 큰 격차를 줄여 줍니다.

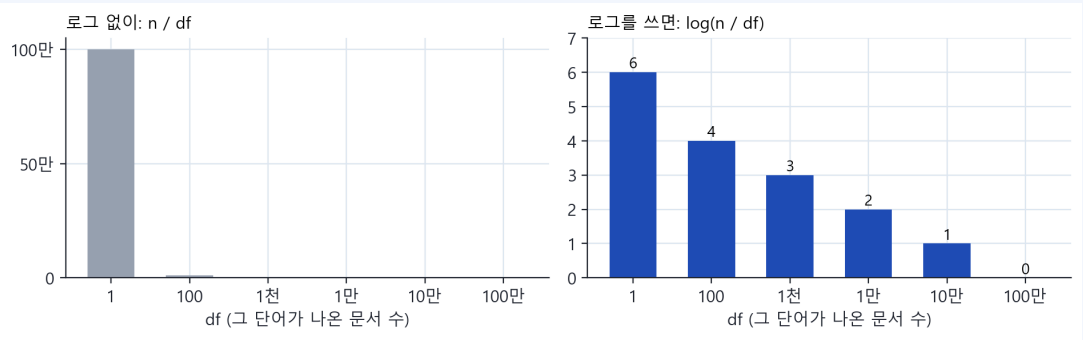


그림 4 문서가 100만 개일 때(밑이 10인 로그). 로그가 없으면 df가 1인 단어 하나가 나머지를 모두 압도하고, 로그를 쓰면 6, 4, 3, 2, 1, 0으로 고르게 줄어든다

이 기본 식의 문제와 사이킷런의 식. 문서가 4개인데 3개에 나온 단어는 $\log(4/4) = 0$이 되어, 나왔는데도 값이 0이 됩니다. 4개 모두에 나오면 음수가 됩니다. 그래서 실제 패키지는 식을 조금 고쳐 씁니다. 사이킷런의 `TfidfVectorizer` 는

$$idf(t) = \ln\left(\frac{1+n}{1+df(t)}\right) + 1$$

분자에도 1을 더하고 결과에 1을 더해 IDF가 항상 1 이상이 되게 합니다. 그리고 문서 벡터마다 길이가 1이 되도록 나눕니다(L2 정규화). 값은 달라져도 "흔한 단어는 낮게, 드문 단어는 높게"라는 뜻은 같습니다.

예시 적용

문서 네 개에 교재의 기본 식(자연로그)을 적용합니다. $n = 4$입니다.

단어	나온 문서	df	$idf = \ln(4/(1+df))$
먹고, 싶은, 바나나	2개씩	2	$\ln(4/3) = 0.288$
사과, 길고, 노란, 저는, 과일이, 좋아요	1개씩	1	$\ln(4/2) = 0.693$

"바나나"는 문서 3에 두 번 나왔지만 나온 문서는 문서 2와 문서 3 두 개이므로 $df = 2$입니다. 이 IDF를 DTM의 값에 곱하면 TF-IDF 행렬이 됩니다.

TF-IDF: DTM의 값에 단어마다의 IDF를 곱한 행렬. 모양 (4, 9)

	과일이	길고	노란	먹고	바나나	사과	싶은	저는	좋아요
문서1: 먹고 싶은 사과	0	0	0	0.29	0	0.69	0.29	0	0
문서2: 먹고 싶은 바나나	0	0	0	0.29	0.29	0	0.29	0	0
문서3: 길고 노란 바나나 바나나	0	0.69	0.69	0	0.58	0	0	0	0
문서4: 저는 과일이 좋아요	0.69	0	0	0	0	0	0	0.69	0.69

진한 칸: 한 문서에만 나온 단어(0.69). 옅은 칸: 두 문서에 나온 단어(0.29). 문서3의 '바나나'는 2 × 0.29 = 0.58

그림 5 문서 1에서 "먹고", "사과", "싶은"은 모두 한 번씩 나왔지만, 문서 1에만 있는 "사과"(0.69)가 다른 문서에도 있는 "먹고", "싶은"(0.29)보다 높다

DTM에서는 문서 1의 세 단어가 모두 1이라 무엇이 이 문서의 특징인지 알 수 없었습니다. TF-IDF에서는 "사과"가 가장 높아 문서 1을 대표하는 단어로 드러납니다. 문서 3의 "바나나"는 TF가 2라서 $2 \times 0.288 = 0.58$로, TF가 1인 문서 2의 "바나나"(0.29)보다 높습니다.

사이킷런으로 영어 문서 세 개의 TF-IDF를 구하고 위의 식으로 직접 계산한 값과 맞춰 보면 같습니다(실습 5). 두 문서에 나온 you의 IDF는 1.288, 한 문서에만 나온 like는 1.693입니다. "I like you"의 문서 벡터에서 like(0.80)가 you(0.61)보다 높게 나옵니다.

비유

반 친구들의 별명 짓기

별명은 그 친구만의 특징으로 짓습니다. "밥을 먹는 애"는 모두에게 해당하므로 별명이 될 수 없고, "매일 기타를 치는 애"는 그 친구만 그래서 별명이 됩니다. 그 친구가 자주 하는 행동이면서(TF), 다른 친구들은 잘 하지 않는 행동(IDF)이 그 친구를 가장 잘 나타냅니다.

그 친구가 자주 하는 행동	= = TF가 큰 단어
다른 친구들은 잘 안 하는 행동	= = IDF가 큰 단어
모두가 하는 행동 (밥 먹기)	= = 불용어 (IDF가 작음)
별명	= = TF-IDF가 가장 큰 단어

실습 4 DTM, IDF, TF-IDF를 식 그대로 계산하기

```python
from math import log

docs = ["먹고 싶은 사과", "먹고 싶은 바나나",
        "길고 노란 바나나 바나나", "저는 과일이 좋아요"]
tokenized = [d.split() for d in docs]                    # 띄어쓰기로 토큰화
vocab = sorted({w for doc in tokenized for w in doc})
N = len(docs)                                            # 문서의 수

def tf(t, d):
    """문서 d에 단어 t가 나온 횟수"""
    return tokenized[d].count(t)

def df(t):
    """단어 t가 나온 문서의 수"""
    return sum(t in doc for doc in tokenized)

def idf(t):
    """df가 클수록(흔할수록) 작아지는 값"""
    return log(N / (1 + df(t)))

def show(title, cell, width=7):
    print(title)
    print("       " + "".join(f"{w:>{width - len(w)}}" for w in vocab))
    for d in range(N):
        print(f"  문서{d + 1}" + "".join(f"{cell(t, d):>{width}}" for t in vocab))

show("DTM (= TF)", lambda t, d: tf(t, d))
print("\ndf :", {t: df(t) for t in vocab})
print("idf:", {t: round(idf(t), 3) for t in vocab})
print()
show("TF-IDF = TF × IDF", lambda t, d: f"{tf(t, d) * idf(t):.2f}")
```

실행 결과

```
DTM (= TF)
        과일이   길고   노란   먹고 바나나   사과   싶은   저는 좋아요
  문서1      0      0      0      1      0      1      1      0      0
  문서2      0      0      0      1      1      0      1      0      0
  문서3      0      1      1      0      2      0      0      0      0
  문서4      1      0      0      0      0      0      0      1      1

df : {'과일이': 1, '길고': 1, '노란': 1, '먹고': 2, '바나나': 2, '사과': 1, '싶은': 2, '저는': 1, '좋아요': 1}
idf: {'과일이': 0.693, '길고': 0.693, '노란': 0.693, '먹고': 0.288, '바나나': 0.288, '사과': 0.693, '싶은':
0.288, '저는': 0.693, '좋아요': 0.693}

TF-IDF = TF × IDF
        과일이   길고   노란   먹고 바나나   사과   싶은   저는 좋아요
  문서1   0.00   0.00   0.00   0.29   0.00   0.69   0.29   0.00   0.00
  문서2   0.00   0.00   0.00   0.29   0.29   0.00   0.29   0.00   0.00
  문서3   0.00   0.69   0.69   0.00   0.58   0.00   0.00   0.00   0.00
  문서4   0.69   0.00   0.00   0.00   0.00   0.00   0.00   0.69   0.69
```

직접 해 보기 문서를 하나 더 추가해 보세요: "먹고 싶은 딸기". "먹고"의 df가 3이 되면 IDF는 $\ln(5/4)$가 됩니다. 문서 1에서 "사과"와 "먹고"의 차이가 더 벌어지나요?

실습 5 사이킷런의 TF-IDF, 그리고 직접 계산해 맞춰 보기

```python
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

corpus = ["you know I want your love", "I like you", "what should I do"]

# 1) DTM
cv = CountVectorizer()
dtm = cv.fit_transform(corpus).toarray()
words = cv.get_feature_names_out()
print("단어:", list(words))
print("DTM:\n", dtm)

# 2) TF-IDF
tv = TfidfVectorizer()
tfidf = tv.fit_transform(corpus).toarray()
print("TF-IDF:\n", tfidf.round(2))

# 3) 사이킷런의 식을 직접 계산해 맞춰 보기
#    idf = ln((1 + n) / (1 + df)) + 1,  그리고 문서 벡터의 길이를 1로 맞춤(L2 정규화)
n = len(corpus)
df = (dtm > 0).sum(axis=0)                      # 단어마다 나온 문서의 수
idf = np.log((1 + n) / (1 + df)) + 1
raw = dtm * idf                                 # TF × IDF
mine = raw / np.linalg.norm(raw, axis=1, keepdims=True)
print("직접 계산 = 사이킷런 결과?", np.allclose(mine, tfidf))

i = list(words).index("you")
print(f"'you': df = {df[i]}, idf = {idf[i]:.3f} / 'like': df = 1,",
      f"idf = {idf[list(words).index('like')]:.3f}")
print("문서 벡터의 길이:", np.linalg.norm(tfidf, axis=1).round(2))
```

실행 결과

```
단어: ['do', 'know', 'like', 'love', 'should', 'want', 'what', 'you', 'your']
DTM:
 [[0 1 0 1 0 1 0 1 1]
 [0 0 1 0 0 0 0 1 0]
 [1 0 0 0 1 0 1 0 0]]
TF-IDF:
 [[0.   0.47 0.   0.47 0.   0.47 0.   0.36 0.47]
 [0.   0.   0.8  0.   0.   0.   0.   0.61 0.  ]
 [0.58 0.   0.   0.   0.58 0.   0.58 0.   0.  ]]
직접 계산 = 사이킷런 결과? True
'you': df = 2, idf = 1.288 / 'like': df = 1, idf = 1.693
문서 벡터의 길이: [1. 1. 1.]
```

직접 해 보기 `TfidfVectorizer(norm=None)`으로 바꾸면 값이 어떻게 달라지나요? 문서 벡터의 길이를 1로 맞추는 것이 다음 절의 코사인 유사도와 어떤 관계가 있을지 생각해 보세요.

확인 문제

11-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 DTM에서 행은 문서, 열은 단어이다. (O / X) 이름 _____

2 모든 문서에 자주 나오는 단어일수록 TF-IDF 값이 커진다. (O / X) 이름 _____

3 $df(t)$가 뜻하는 것은? 이름 _____

① 단어 t가 나온 총 횟수 ② 단어 t가 나온 문서의 수 ③ 문서의 총 개수

4 IDF에 로그를 씌우는 이유는? 이름 _____

① 희귀한 단어의 값이 지나치게 커지는 것을 막으려고 ② 계산을 빠르게 하려고 ③ 값을 음수로 만들려고

5 대부분의 값이 0인 행렬을 _____ 행렬이라고 한다. 이름 _____

6 실습에서 문서 1의 "사과"의 TF-IDF는 0.69, "먹고"는 _____였다. 이름 _____

7 TF, DF, IDF를 각각 정의하고, 문서 1에서 "사과"가 "먹고"보다 TF-IDF가 높은 이유를 설명해 보세요. 이름 _____

원문: 11-03 DTM과 TF-IDF 행렬 — https://wikidocs.net/217501

11-04 코사인 유사도를 이용한 추천 시스템

핵심 한 줄

코사인 유사도는 두 벡터가 가리키는 방향이 얼마나 같은지를 재는 값이고, 문서 벡터에 쓰면 길이에 상관없이 내용이 비슷한 문서를 찾을 수 있다.

왜 배우나

문서를 벡터로 바꿨으니 이제 "두 벡터가 얼마나 비슷한가"를 숫자 하나로 계산할 차례입니다. 이 숫자만 있으면 가장 비슷한 문서를 찾을 수 있고, 그것이 검색과 추천의 기본입니다. 코사인 유사도는 12장의 워드 임베딩, 뒤에 나올 문장 임베딩과 검색에서도 계속 쓰입니다.

1) 코사인 유사도

정의

코사인 유사도(cosine similarity)는 두 벡터 사이의 각도 θ의 코사인 값입니다.

$$\text{similarity} = \cos\theta = \frac{A \cdot B}{\|A\|\,\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\,\sqrt{\sum_{i=1}^{n} B_i^2}}$$

분자 $A \cdot B$는 내적으로, 같은 자리의 값끼리 곱해 모두 더한 것입니다. 분모의 $\|A\|$는 벡터의 길이로, 값을 제곱해 더한 뒤 제곱근을 씌운 것입니다. 말로 하면: "같은 칸에서 함께 큰 정도"를 두 벡터의 길이로 나눈 값.

- 값의 범위는 -1부터 1까지입니다. 방향이 완전히 같으면 1, 직각이면 0, 정반대면 -1입니다. **1에 가까울수록 비슷합니다.**
- DTM과 TF-IDF는 값이 0 이상이므로 문서 벡터끼리의 코사인 유사도는 0과 1 사이입니다. 겹치는 단어가 하나도 없으면 0입니다.

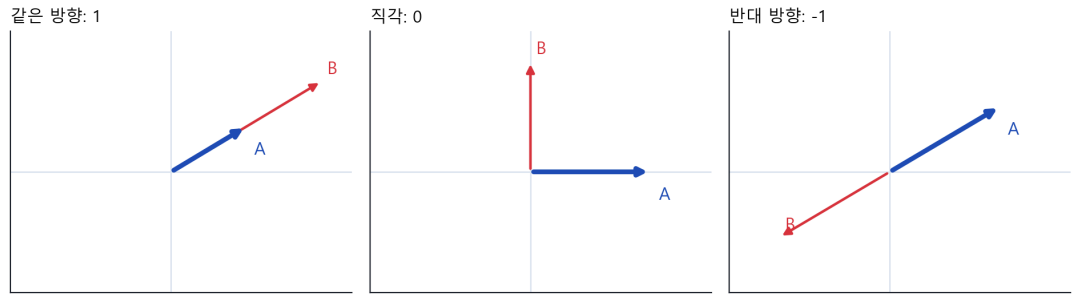


그림 6 코사인 유사도는 길이를 보지 않고 방향만 본다

왜 길이로 나눌까? 내적만 쓰면 긴 문서가 무조건 유리합니다. 단어가 많으니 값이 크고, 값이 크면 내적도 커집니다. 길이로 나누면 문서의 **길이는 지워지고 단어들의 비율(방향)만 남습니다.** 같은 주제를 짧게 쓴 글과 길게 쓴 글을 비슷하다고 판단할 수 있게 됩니다.

예시 적용

세 문서를 DTM으로 만들면(단어 순서: 바나나, 사과, 저는, 좋아요) 다음과 같습니다(실습 6).

문서	내용	문서 벡터
문서 1	저는 사과 좋아요	[0, 1, 1, 1]
문서 2	저는 바나나 좋아요	[1, 0, 1, 1]
문서 3	저는 바나나 좋아요 저는 바나나 좋아요	[2, 0, 2, 2]

문서 1과 문서 2를 식에 넣습니다. 내적은 $0\times1+1\times0+1\times1+1\times1=2$이고, 두 벡터의 길이는 모두 $\sqrt{1^2+1^2+1^2}=\sqrt{3}=1.732$입니다. 그래서 $2/(1.732\times1.732)=0.67$이고, 각도로는 48.2도입니다.

비교	코사인 유사도	해석
문서 1 - 문서 2	0.67	"저는", "좋아요"는 겹치고 과일만 다름
문서 1 - 문서 3	0.67	문서 2와의 유사도와 같다
문서 2 - 문서 3	1.00	문서 3은 문서 2를 두 번 쓴 것. 길이만 2배이고 방향이 같다

문서 3은 문서 2보다 값이 모두 2배지만 유사도는 1입니다. 문서 1에서 보면 문서 2와 문서 3이 똑같이 0.67만큼 비슷합니다. 길이가 결과에 영향을 주지 않는다는 것을 확인할 수 있습니다.

2) 유사도를 이용한 추천

정의

콘텐츠 기반 추천은 사용자가 좋아한 항목과 내용이 비슷한 항목을 골라 주는 방법입니다. 텍스트라면 지금까지 배운 것만으로 만들 수 있습니다.

$$\text{문서 } n\text{개} \xrightarrow{\text{TF-IDF}} \text{행렬 } (n,\ V) \xrightarrow{\text{모든 쌍의 코사인 유사도}} \text{행렬 } (n,\ n) \xrightarrow{\text{한 행을 큰 순서로 정렬}} \text{추천 목록}$$

유사도 행렬의 i행 j열은 문서 i와 문서 j의 유사도입니다. 대각선은 자기 자신과의 유사도라서 항상 1이고, 추천할 때는 빼야 합니다.

예시 적용

교재는 캐글의 영화 2만 편 줄거리로 추천 시스템을 만듭니다. TF-IDF 행렬이 (20000, 47487), 유사도 행렬이 (20000, 20000)이고, "다크 나이트 라이즈"를 넣으면 배트맨 영화 10편이 나옵니다. 같은 방법을 영화 8편의 한 줄 줄거리로 해 봅니다(실습 7). TF-IDF 행렬은 (8, 46), 유사도 행렬은 (8, 8)입니다.

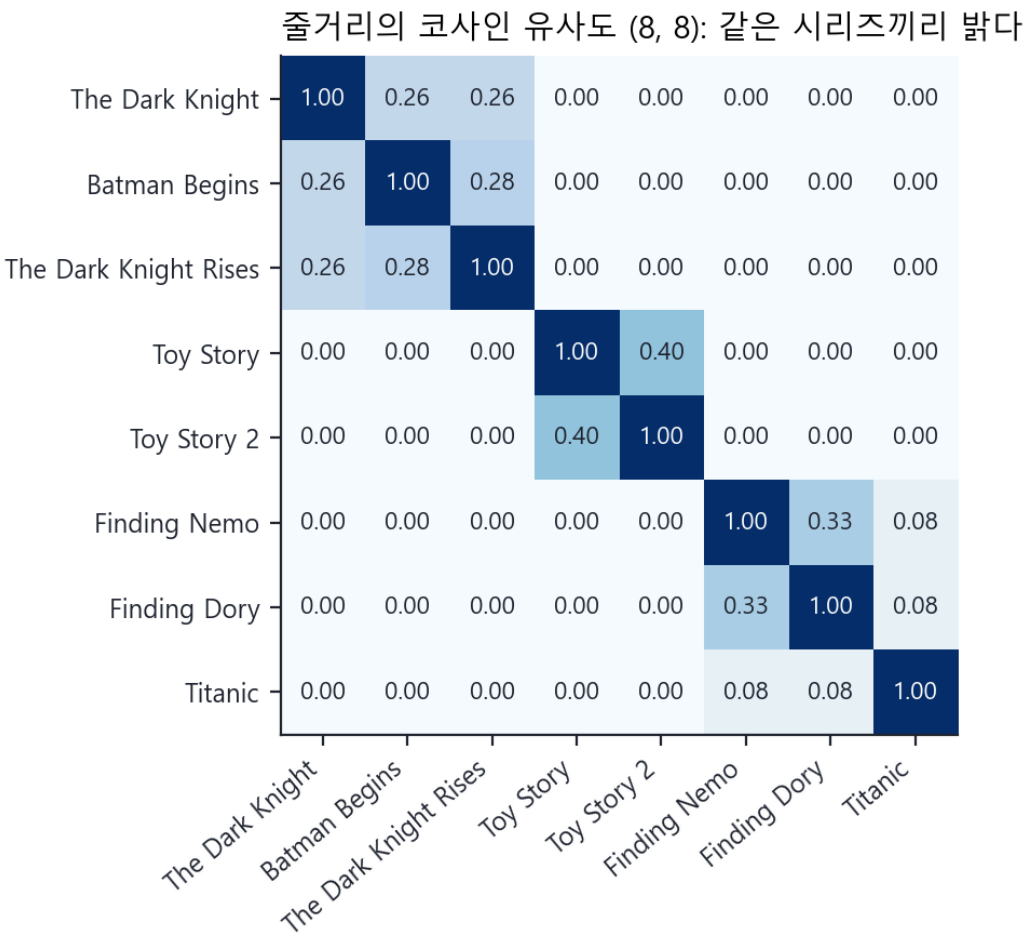


그림 7 영화 8편의 줄거리 유사도. 배트맨 세 편, 토이 스토리 두 편, 물고기 영화 두 편이 서로 밝게 묶인다

- "The Dark Knight Rises"를 넣으면 Batman Begins(0.28), The Dark Knight(0.26)가 추천됩니다. 줄거리에 Batman, Gotham, city가 함께 나오기 때문입니다.

- "Toy Story"를 넣으면 Toy Story 2(0.40)가 나옵니다. toy와 Woody가 겹칩니다.

틀리는 경우. "Titanic"을 넣으면 Finding Dory와 Finding Nemo가 0.08로 추천됩니다. 로맨스 영화에 물고기 애니메이션이 추천된 이유는 줄거리에 ocean이라는 단어가 함께 나오기 때문입니다. 이 방법은 **같은 단어가 나왔는지만** 보고, 내용이 비슷한지는 모릅니다. 반대로 ship과 boat처럼 뜻이 비슷해도 글자가 다르면 전혀 겹치지 않는 것으로 봅니다. 11-01에서 본 국소 표현의 한계이고, 12장의 워드 임베딩이 풀려는 문제입니다.

비유

음료의 배합 비율

작은 컵에 탄 "커피 1 : 우유 2"와 큰 통에 탄 "커피 10 : 우유 20"은 양은 10배 차이지만 같은 맛입니다. 맛을 정하는 것은 양이 아니라 비율입니다. 코사인 유사도는 양(벡터의 길이)을 무시하고 배합 비율(방향)이 같은지를 봅니다.

재료마다의 양	= = 문서 벡터의 값 (단어의 횟수)
전체 양	= = 벡터의 길이 (문서의 길이)
배합 비율	= = 벡터의 방향
같은 맛인가	= = 코사인 유사도

실습 6 코사인 유사도를 식 그대로 계산하기

```python
import numpy as np

def cos_sim(A, B):
    """코사인 유사도 = 내적 / (길이 × 길이)"""
    return A @ B / (np.linalg.norm(A) * np.linalg.norm(B))

# 단어 순서: [바나나, 사과, 저는, 좋아요]
doc1 = np.array([0, 1, 1, 1])      # 저는 사과 좋아요
doc2 = np.array([1, 0, 1, 1])      # 저는 바나나 좋아요
doc3 = np.array([2, 0, 2, 2])      # 저는 바나나 좋아요 저는 바나나 좋아요

# 식을 단계별로: 문서1과 문서2
dot = doc1 @ doc2
len1, len2 = np.linalg.norm(doc1), np.linalg.norm(doc2)
print(f"내적 = {dot}, 길이 = {len1:.3f}, {len2:.3f}",
      f"→ {dot} / ({len1:.3f} × {len2:.3f}) = {dot / (len1 * len2):.2f}")

print("문서1-문서2:", round(cos_sim(doc1, doc2), 2))
print("문서1-문서3:", round(cos_sim(doc1, doc3), 2))
print("문서2-문서3:", round(cos_sim(doc2, doc3), 2), " ← 길이만 2배, 방향은 같다")

# 각도로 보면
for name, a, b in [("문서1-문서2", doc1, doc2), ("문서2-문서3", doc2, doc3)]:
    angle = np.degrees(np.arccos(np.clip(cos_sim(a, b), -1, 1)))
    print(f"  {name} 사이의 각도: {angle:.1f}도")
```

```
실행 결과
내적 = 2, 길이 = 1.732, 1.732 → 2 / (1.732 × 1.732) = 0.67
문서1-문서2: 0.67
문서1-문서3: 0.67
문서2-문서3: 1.0  ← 길이만 2배, 방향은 같다
  문서1-문서2 사이의 각도: 48.2도
  문서2-문서3 사이의 각도: 0.0도
```

직접 해 보기 문서 4 "저는 포도 싫어요"를 [바나나, 사과, 저는, 좋아요, 포도, 싫어요] 순서의 벡터로 만들어 문서 1과의 유사도를 구해 보세요 (다른 문서에도 0 두 칸을 붙여야 합니다).

실습 7 영화 줄거리로 비슷한 영화 추천하기

```python
# 교재는 캐글의 영화 2만 편 줄거리(movies_metadata.csv)를 쓴다.
# 여기서는 같은 방법을 영화 8편의 짧은 줄거리로 한다.
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

movies = {
    "The Dark Knight":
        "Batman fights the Joker, a criminal who brings chaos to Gotham city.",
    "Batman Begins":
        "Bruce Wayne becomes Batman to fight crime in Gotham city.",
    "The Dark Knight Rises":
        "Batman returns to save Gotham city from the terrorist Bane.",
```

```
    "Toy Story":
        "A cowboy toy named Woody feels jealous when a new space toy arrives.",
    "Toy Story 2":
        "Woody is stolen by a toy collector and his toy friends rescue him.",
    "Finding Nemo":
        "A clownfish father crosses the ocean to find his lost son Nemo.",
    "Finding Dory":
        "The forgetful fish Dory crosses the ocean to find her lost parents.",
    "Titanic":
        "A young couple fall in love on a ship that sinks in the ocean.",
}
titles = list(movies)

# 1) 줄거리 → TF-IDF 행렬
tfidf = TfidfVectorizer(stop_words="english")
matrix = tfidf.fit_transform(movies.values())
print("TF-IDF 행렬:", matrix.shape, "= (영화 수, 단어 수)")

# 2) 모든 영화 쌍의 코사인 유사도
cosine_sim = cosine_similarity(matrix, matrix)
print("유사도 행렬:", cosine_sim.shape)

def recommend(title, k=3):
    """title과 줄거리가 비슷한 영화를 k편 고른다"""
    idx = titles.index(title)
    scores = sorted(enumerate(cosine_sim[idx]), key=lambda x: x[1], reverse=True)
    return [(titles[i], round(s, 2)) for i, s in scores[1:k + 1]]   # 0번은 자기 자신

for t in ["The Dark Knight Rises", "Toy Story", "Titanic"]:
    print(f"\n'{t}'와 비슷한 영화:")
    for name, score in recommend(t):
        print(f"   {score:.2f}  {name}")
```

실행 결과

```
TF-IDF 행렬: (8, 46) = (영화 수, 단어 수)
유사도 행렬: (8, 8)

'The Dark Knight Rises'와 비슷한 영화:
   0.28  Batman Begins
   0.26  The Dark Knight
   0.00  Toy Story

'Toy Story'와 비슷한 영화:
   0.40  Toy Story 2
   0.00  The Dark Knight
   0.00  Batman Begins

'Titanic'와 비슷한 영화:
   0.08  Finding Dory
   0.08  Finding Nemo
   0.00  The Dark Knight
```

직접 해 보기 Titanic의 줄거리에서 "in the ocean"을 지우고 다시 실행해 보세요. 추천 결과가 어떻게 바뀌나요? 단어 하나가 추천을 좌우한다는 점을 확인해 보세요.

11-04 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 코사인 유사도는 1에 가까울수록 두 벡터가 비슷하다. (O / X) 이름 _____

2 문서의 모든 단어 횟수가 2배가 되면 코사인 유사도도 2배가 된다. (O / X) 이름 _____

3 두 문서에 겹치는 단어가 하나도 없을 때 DTM으로 잰 코사인 유사도는? 이름 _____

① -1 ② 0 ③ 1

4 유사도 행렬의 대각선 값이 모두 1인 이유는? 이름 _____

① 자기 자신과의 유사도라서 ② 계산 오류라서 ③ 불용어를 지워서

5 코사인 유사도의 분자는 두 벡터의 _____이다. 이름 _____

6 실습에서 문서 2와 문서 3의 코사인 유사도는 _____이었다. 이름 _____

7 코사인 유사도를 정의하고, 실습 7에서 Titanic에 물고기 영화가 추천된 이유를 설명해 보세요. 이름 _____

원문: 11-04 코사인 유사도를 이용한 추천 시스템 — https://wikidocs.net/217505 (교재의 캐글 영화 데이터 2만 편을 직접 쓴 한 줄 줄거리 8편으로 줄여 설명했습니다. 큰 데이터는 코랩에서 교재 코드로 돌려 보세요)

11-05 단어와 문서의 유사도를 구하는 다양한 방법

핵심 한 줄 **유클리드 거리는 두 점 사이의 직선 거리를, 자카드 유사도는 겹치는 단어의 비율을 재며, 무엇을 "비슷하다"고 볼지에 따라 방법을 고른다.**

왜 배우나 "비슷하다"의 뜻은 하나가 아닙니다. 방향이 같은 것, 거리가 가까운 것, 같은 단어를 많이 쓴 것은 서로 다른 기준입니다. 세 방법이 같은 문서에 다른 답을 내는 것을 보면, 코사인 유사도가 왜 텍스트에서 기본으로 쓰이는지도 분명해집니다.

1) 유클리드 거리

정의

유클리드 거리(Euclidean distance)는 두 점 $p = (p_1, \dots, p_n)$, $q = (q_1, \dots, q_n)$ 사이의 직선 거리입니다.

$$d(p, q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \cdots + (q_n - p_n)^2} = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

칸마다 차이를 구해 제곱하고, 모두 더한 뒤 제곱근을 씌웁니다. 2차원이면 피타고라스의 정리로 구하는 빗변의 길이이고, 그것을 단어의 수만큼의 차원으로 늘린 것입니다.

- 유사도가 아니라 **거리**이므로 **작을수록 비슷합니다.** 0이면 완전히 같습니다.
- 값의 크기를 그대로 반영합니다. 그래서 내용이 같아도 길이가 다른 문서는 멀다고 판단합니다.

예시 적용 단어 4개(바나나, 사과, 저는, 좋아요)로 만든 DTM에서 문서 Q = [1, 1, 0, 1]과 가장 가까운 문서를 찾습니다(실습 8).

문서	벡터	문서 Q와의 거리
문서 1	[2, 3, 0, 1]	$\sqrt{1^2 + 2^2 + 0^2 + 0^2} = \sqrt{5} = 2.24$
문서 2	[1, 2, 3, 1]	$\sqrt{0^2 + 1^2 + 3^2 + 0^2} = \sqrt{10} = 3.16$
문서 3	[2, 1, 2, 2]	$\sqrt{1^2 + 0^2 + 2^2 + 1^2} = \sqrt{6} = 2.45$

거리가 가장 작은 문서 1이 문서 Q와 가장 비슷하다고 판단합니다.

2) 자카드 유사도

정의

자카드 유사도(Jaccard similarity)는 두 집합 A, B에서 **합집합 중 교집합이 차지하는 비율**입니다.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

$A \cap B$는 교집합(둘 다에 있는 것), $A \cup B$는 합집합(둘 중 하나에라도 있는 것), $|\cdot|$는 원소의 개수입니다. 문서에 쓰면 "두 문서에 나온 단어를 통틀어, 둘 다에 나온 단어의 비율"입니다.

- 값의 범위는 0부터 1까지입니다. 두 집합이 같으면 1, 겹치는 것이 없으면 0입니다.
- 집합으로 다루므로 단어가 **나왔는지만** 보고 몇 번 나왔는지는 보지 않습니다.

예시 적용 문서 1 "apple banana everyone like likey watch card holder"(8단어)와 문서 2 "apple banana coupon passport love you"(6단어)를 비교합니다(실습 8). 두 문서에 나온 단어를 모두 모으면 12개이고, 둘 다에 나온 단어는 apple과 banana 2개입니다. 그래서 $2/12 = 0.17$입니다.

3) 세 방법의 비교

정의

	코사인 유사도	유클리드 거리	자카드 유사도
재는 것	두 벡터의 방향이 같은 정도	두 점 사이의 직선 거리	겹치는 원소의 비율
입력	벡터	벡터	집합
범위	-1 ~ 1 (횟수 벡터는 0 ~ 1)	0 이상	0 ~ 1
비슷할수록	커진다 (1)	작아진다 (0)	커진다 (1)
단어의 횟수	비율로 반영	크기 그대로 반영	반영하지 않음
문서의 길이	영향 없음	영향을 크게 받음	영향 없음 (나온 단어만 봄)

문서는 같은 주제라도 길이가 크게 다를 수 있습니다. 그래서 문서의 유사도에는 길이에 영향을 받지 않는 코사인 유사도를 가장 많이 씁니다.

예시 적용

"저는 바나나 좋아요" [1, 0, 1, 1]과, 같은 문장을 두 번 쓴 [2, 0, 2, 2]를 세 방법으로 비교합니다(실습 8). 내용은 완전히 같고 길이만 2배입니다.

방법	값	판단
코사인 유사도	1.0	완전히 같다
유클리드 거리	1.73	떨어져 있다 (0이 아님)
자카드 유사도	1.0	완전히 같다

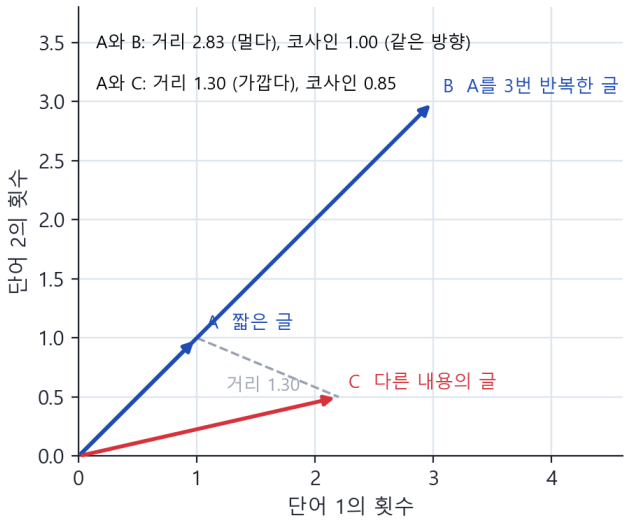


그림 8 단어 두 개만 있는 경우를 그린 것. A를 3번 반복한 B는 A와 방향이 같지만 거리는 멀다. 내용이 다른 C가 거리로는 오히려 A에 더 가깝다

유클리드 거리만 보면 "A는 자신을 반복한 B보다 내용이 다른 C와 더 비슷하다"는 이상한 결론이 나옵니다. 문서 비교에 유클리드 거리를 잘 쓰지 않는 이유입니다.

비유

두 사람의 플레이리스트 비교하기

두 사람의 음악 취향이 비슷한지 보는 방법은 여러 가지입니다. 장르별로 듣는 비율이 비슷한지 볼 수도 있고, 장르별 재생 횟수의 차이를 볼 수도 있고, 같은 곡이 몇 곡이나 겹치는지 볼 수도 있습니다. 하루에 10곡 듣는 사람과 100곡 듣는 사람을 비교한다면, 재생 횟수의 차이보다는 비율을 보는 편이 맞습니다.

장르별 비율이 비슷한가 = = 코사인 유사도
장르별 재생 횟수의 차이 = = 유클리드 거리
겹치는 곡의 비율 = = 자카드 유사도

실습 8

유클리드 거리, 자카드 유사도, 세 방법 비교

```python
import numpy as np

# 1) 유클리드 거리: 두 점 사이의 직선 거리 (작을수록 비슷)
def dist(x, y):
    return np.sqrt(np.sum((x - y) ** 2))

# 단어 순서: [바나나, 사과, 저는, 좋아요]
doc1 = np.array([2, 3, 0, 1])
doc2 = np.array([1, 2, 3, 1])
doc3 = np.array([2, 1, 2, 2])
docQ = np.array([1, 1, 0, 1])
for name, d in [("문서1", doc1), ("문서2", doc2), ("문서3", doc3)]:
    print(f"{name}과 문서Q의 거리: {dist(d, docQ):.2f}")

# 2) 자카드 유사도: 겹치는 단어의 수 / 전체 단어의 수 (클수록 비슷)
a = "apple banana everyone like likey watch card holder".split()
b = "apple banana coupon passport love you".split()
union = set(a) | set(b)                # 합집합: 둘 중 하나에라도 있는 단어
inter = set(a) & set(b)                # 교집합: 둘 다에 있는 단어
print("\n합집합:", len(union), "개 / 교집합:", sorted(inter))
jaccard = len(inter) / len(union)
print("자카드 유사도:", len(inter), "/", len(union), "=", round(jaccard, 2))

# 3) 같은 두 문서를 세 방법으로: 내용은 같고 길이만 2배인 경우
def cos_sim(A, B):
    return A @ B / (np.linalg.norm(A) * np.linalg.norm(B))

short = np.array([1, 0, 1, 1])         # 저는 바나나 좋아요
long = np.array([2, 0, 2, 2])          # 같은 문장을 두 번
print("\n같은 내용, 길이만 2배인 두 문서")
print("  코사인 유사도:", round(cos_sim(short, long), 2))
print("  유클리드 거리:", round(dist(short, long), 2))
words = lambda v: {i for i, c in enumerate(v) if c > 0}       # 나온 단어의 집합
s, l = words(short), words(long)
print("  자카드 유사도:", len(s & l) / len(s | l))
```

실행 결과

```
문서1과 문서Q의 거리: 2.24
문서2과 문서Q의 거리: 3.16
문서3과 문서Q의 거리: 2.45

합집합: 12 개 / 교집합: ['apple', 'banana']
자카드 유사도: 2 / 12 = 0.17

같은 내용, 길이만 2배인 두 문서
  코사인 유사도: 1.0
  유클리드 거리: 1.73
  자카드 유사도: 1.0
```

직접 해 보기 문서 Q와 문서 1, 2, 3의 **코사인 유사도**도 구해 보세요. 유클리드 거리로 고른 "가장 비슷한 문서"와 같은가요?

11-05 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 유클리드 거리는 값이 클수록 두 문서가 비슷하다. (O / X) 이름 _____

2 자카드 유사도는 단어가 몇 번 나왔는지도 반영한다. (O / X) 이름 _____

3 문서의 길이에 가장 크게 영향을 받는 방법은? 이름 _____

① 코사인 유사도 ② 유클리드 거리 ③ 자카드 유사도

4 자카드 유사도의 분모는? 이름 _____

① 교집합의 크기 ② 합집합의 크기 ③ 두 벡터의 길이의 곱

5 2차원에서 유클리드 거리는 _____의 정리로 구하는 빗변의 길이와 같다. 이름 _____

6 실습에서 두 문서의 합집합은 12개, 교집합은 2개여서 자카드 유사도는 _____이었다. 이름 _____

7 같은 문장을 두 번 쓴 문서를 세 방법으로 비교한 결과를 말하고, 문서 비교에 코사인 유사도를 많이 쓰는 이유를 설명해 보세요. 이름 _____

원문: 11-05 단어와 문서의 유사도를 구하는 다양한 방법 — https://wikidocs.net/217508 (세 방법의 비교는 이 자료에서 추가했습니다)

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
국소 표현 / 분산 표현	단어 자체에 값을 붙임 / 주변 단어를 참고해 뜻을 벡터에 담음	원-핫 유사도 0.0 / 강아지-고양이 0.99
Bag of Words	순서를 버리고 단어의 횟수만 센 벡터. 길이 V	[1, 2, 1, 1, 2, 1, 1, 1, 1, 1]
DTM	문서마다의 BoW를 쌓은 행렬 (n, V)	과일 문서 (4, 9). 36칸 중 24칸이 0
TF / DF / IDF	문서 안의 횟수 / 나온 문서의 수 / $\log(n/(1+df))$	바나나: df 2, idf 0.288
TF-IDF	TF × IDF. 그 문서에서만 자주 나오는 단어가 큼	문서 1: 사과 0.69 > 먹고 0.29
코사인 유사도	내적 ÷ (길이 × 길이). 방향이 같은 정도	0.67, 0.67, 1.00
유클리드 거리	두 점 사이의 직선 거리. 작을수록 비슷	2.24, 3.16, 2.45
자카드 유사도	교집합의 크기 ÷ 합집합의 크기	2 / 12 = 0.17
횟수 세기의 한계	희소 표현, 순서를 모름, 뜻을 모름	Titanic에 물고기 영화 추천 (ocean)

다음

다음 자료 예고: 12장 텍스트 임베딩

오늘의 방법은 같은 글자의 단어가 나왔는지만 봅니다. ship과 boat가 비슷한 말이라는 것도, ocean이 나왔다고 같은 종류의 영화가 아니라는 것도 모릅니다. 12장에서는 단어의 뜻을 수십~수백 칸의 벡터에 담는 워드 임베딩을 배웁니다. 11-01에서 설명용으로 직접 정했던 분산 표현의 값을, 글에서 학습해 얻는 방법입니다.

정답과 해설

11-01 단어의 표현 방법

1. O
2. X — 단어마다 따로 값을 정하므로 값에 뜻이 들어 있지 않습니다.
3. ②
4. ①
5. 이산
6. 0 (0.0)
7. 예시 답: 국소 표현은 단어 자체만 보고 값을 붙이는 방법, 분산 표현은 주변 단어를 참고해 뜻을 벡터의 여러 칸에 나눠 담는 방법이다. 원-핫 벡터에서는 강아지-고양이, 강아지-자동차의 유사도가 모두 0이지만, 분산 표현에서는 강아지-고양이가 0.99로 훨씬 가깝게 나온다.

11-02 백 오브 워즈

1. X — 순서는 버리고 횟수만 셉니다.
2. X — 띄어쓰기로만 잘라서 "물가상승률과", "물가상승률은"을 다른 단어로 셉니다.
3. ②
4. ①
5. 2
6. 2 — 그래서 한 글자인 I가 빠졌습니다.
7. 예시 답: BoW는 단어의 순서를 버리고 횟수만 센 벡터다. 영수증처럼 무엇이 몇 개인지만 봐도 문서의 성격을 짐작할 수 있고 문서끼리 비교할 수 있다(장점). 하지만 담은 순서가 없어서 "dog bites man"과 "man bites dog"를 구분하지 못한다(한계).

11-03 DTM과 TF-IDF 행렬

1. O
2. X — 여러 문서에 나올수록 IDF가 작아져 값이 낮아집니다.
3. ②
4. ① — 100만과 1,000의 차이가 6과 3의 차이로 줄어듭니다.
5. 희소
6. 0.29 (0.288)
7. 예시 답: TF는 문서 안에서 단어가 나온 횟수, DF는 그 단어가 나온 문서의 수, IDF는 DF가 클수록 작아지는 값이다. "사과"와 "먹고"는 문서 1에서 TF가 1로 같지만, "사과"는 문서 1에만 나와 IDF가 0.693이고 "먹고"는 두 문서에 나와 0.288이다. 그래서 "사과"가 더 높다.

11-04 코사인 유사도를 이용한 추천 시스템

1. O
2. X — 방향이 같으므로 그대로입니다. 문서 2와 문서 3의 유사도가 1이었습니다.
3. ② — 내적이 0이 됩니다.
4. ①
5. 내적
6. 1 (1.00)
7. 예시 답: 코사인 유사도는 두 벡터의 내적을 두 벡터의 길이의 곱으로 나눈 값으로, 방향이 같은 정도를 나타낸다. Titanic과 물고기 영화의 줄거리에 ocean이라는 단어가 함께 있어서 유사도가 0.08로 나왔다. 이 방법은 같은 단어가 나왔는지만 보고 내용이 비슷한지는 모른다.

11-05 단어와 문서의 유사도를 구하는 다양한 방법

1. X — 거리이므로 작을수록 비슷합니다.
2. X — 집합으로 다루므로 나왔는지만 봅니다.
3. ②
4. ②
5. 피타고라스
6. 0.17 (2/12)
7. 예시 답: 코사인 유사도 1.0, 유클리드 거리 1.73, 자카드 유사도 1.0이었다. 내용이 같은데도 유클리드 거리는 0이 아니다. 문서는 같은 주제라도 길이가 다를 수 있으므로, 길이에 영향을 받지 않고 단어의 비율을 보는 코사인 유사도를 많이 쓴다.

용어 정리

용어	정의
국소 표현 (이산 표현)	단어 자체만 보고 값을 대응시키는 표현 (번호, 원-핫, BoW)
분산 표현 (연속 표현)	주변 단어를 참고해 뜻을 벡터의 여러 칸에 담는 표현 (워드 임베딩)
Bag of Words (BoW)	단어의 순서를 버리고 횟수만 센 벡터
문서 단어 행렬 (DTM)	문서마다의 BoW를 쌓은 행렬. 행 = 문서, 열 = 단어
문서 벡터	DTM이나 TF-IDF 행렬의 한 행. 문서 하나를 나타내는 벡터
희소 벡터 / 희소 행렬	대부분의 값이 0인 벡터 / 행렬
TF / DF / IDF	문서 안에서 나온 횟수 / 나온 문서의 수 / DF에 반비례하는 값
TF-IDF	TF × IDF. 단어가 그 문서에서 얼마나 중요한지를 나타내는 값
내적	두 벡터의 같은 자리 값끼리 곱해 모두 더한 것
코사인 유사도	두 벡터 사이 각도의 코사인. 방향이 같은 정도 (-1 ~ 1)
유클리드 거리	두 점 사이의 직선 거리
자카드 유사도	두 집합의 교집합 크기 ÷ 합집합 크기 (0 ~ 1)